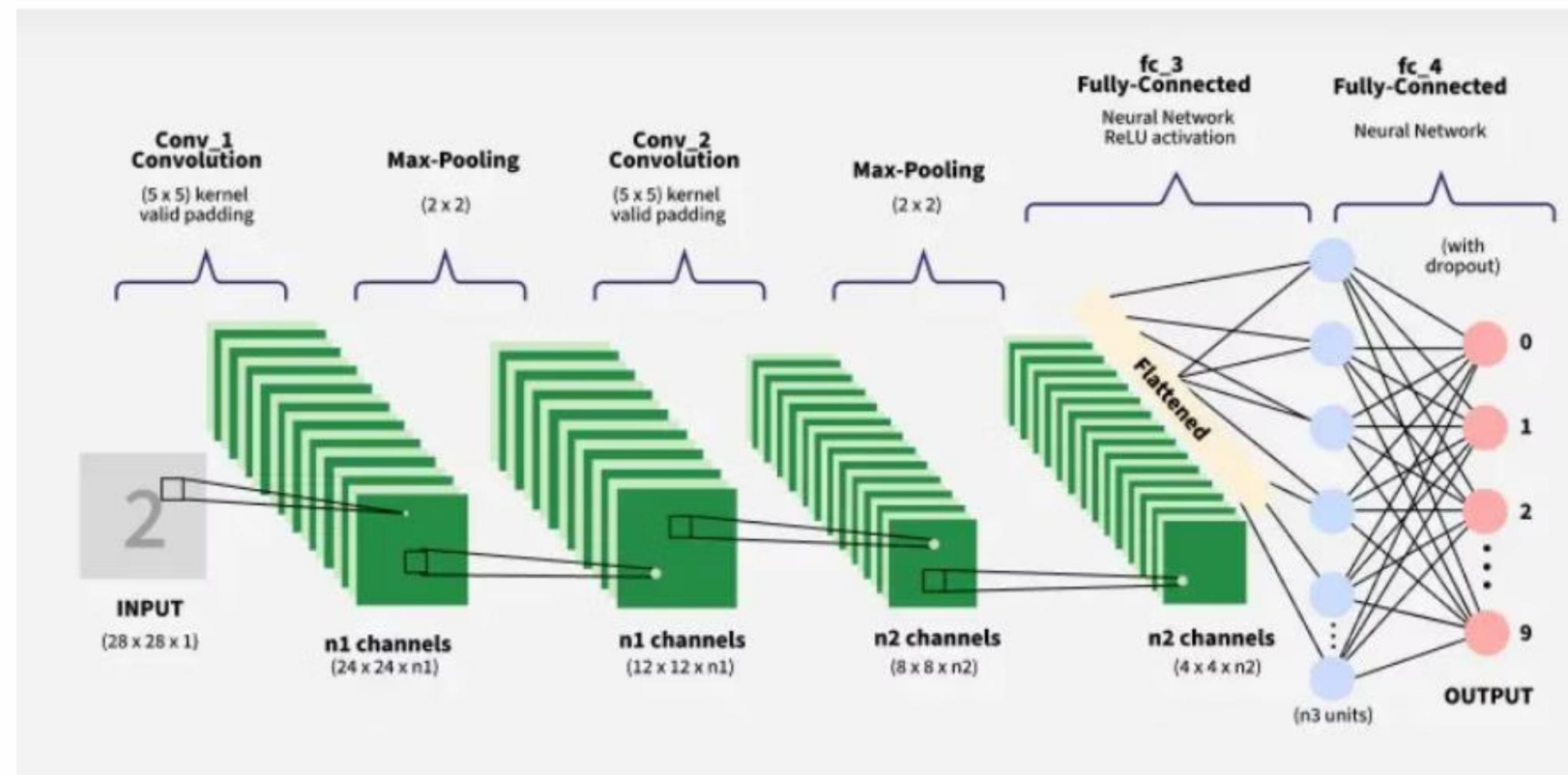


# Convolutional Neural Networks (CNNs)

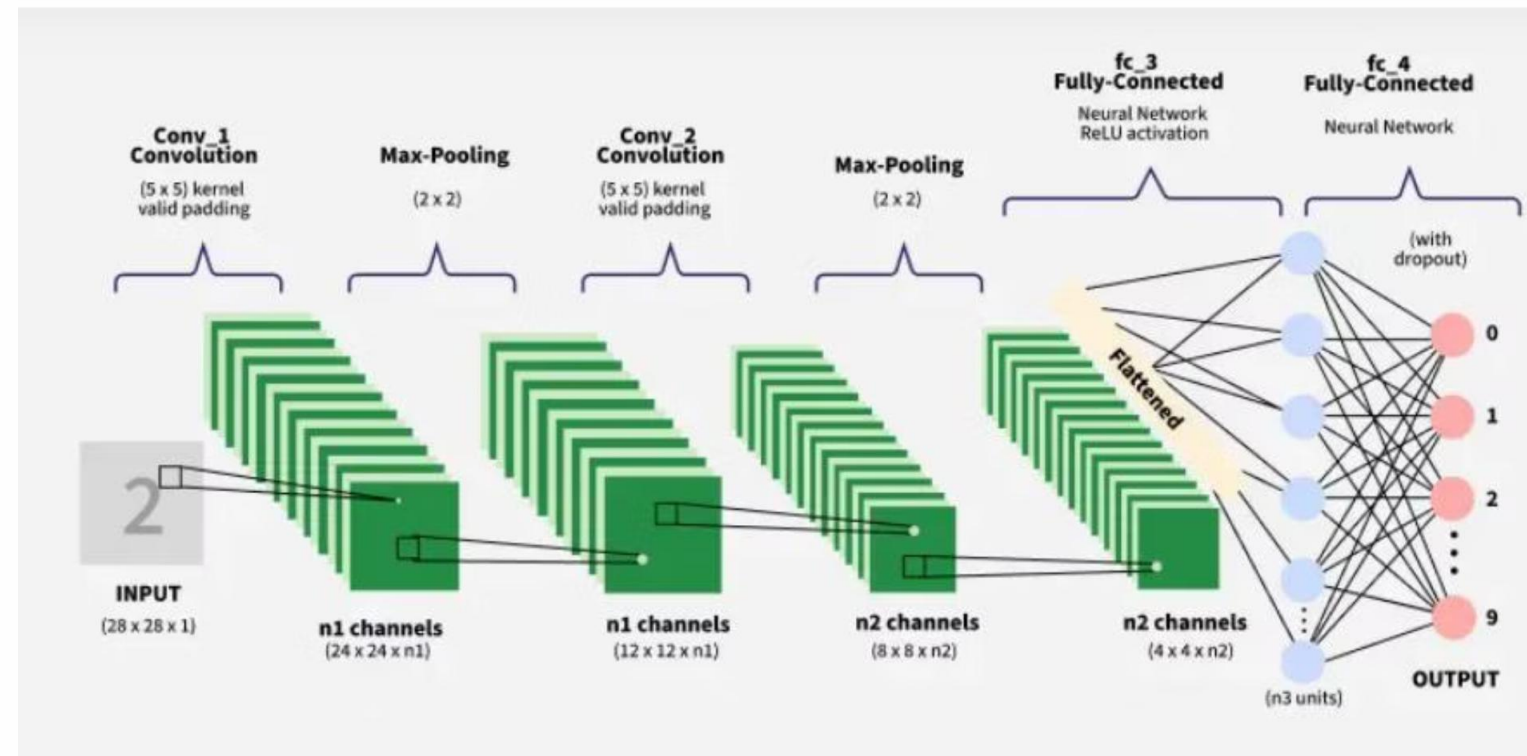
A Convolutional Neural Network (CNN) is a type of deep learning model designed primarily for image processing and computer vision tasks. It automatically learns spatial hierarchies of features (edges, textures, shapes, objects) from images. Unlike traditional neural networks, which treat all input features equally, CNNs exploit the spatial structure of images using convolutional layers.



# CNN

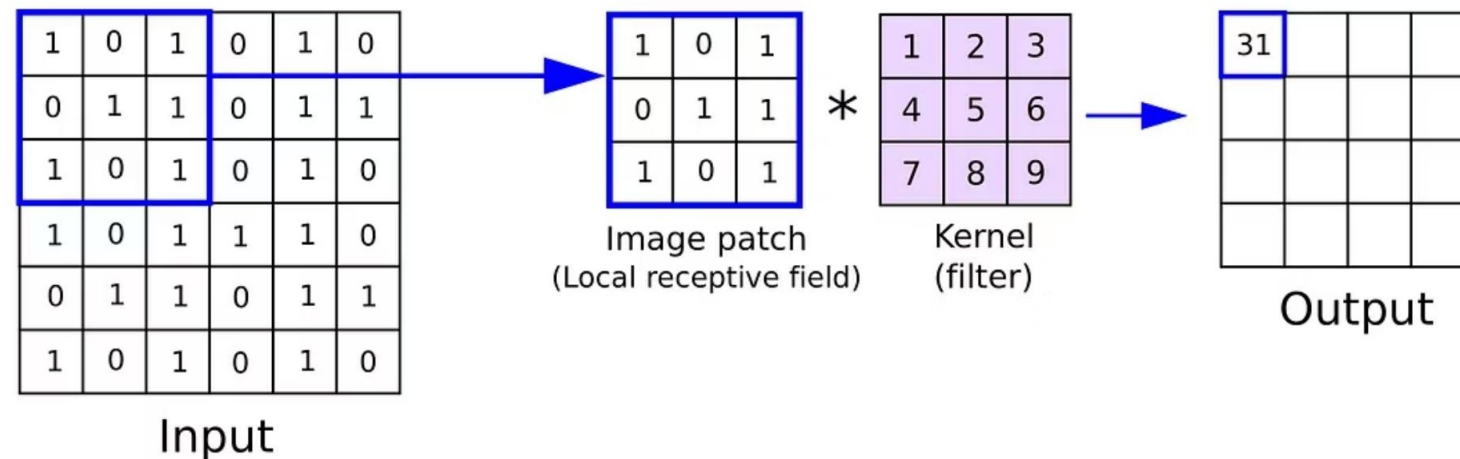
Convolutional neural networks are distinguished from other neural networks by their superior performance with image, speech, or audio signal inputs. They have three main types of layers, which are:

- Convolutional layer
- Pooling layer
- Fully-connected (FC) layer

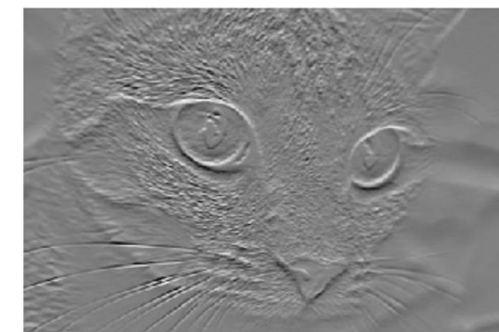


# Input Layer

- The input to a CNN is typically a 2D image (e.g., 32×32, 224×224 pixels) with 1 or 3 channels (grayscale or RGB).
- Each pixel value represents intensity, ranging from 0 to 255 (normalized between 0 and 1 or -1 to 1).



$$\begin{matrix} * \\ \begin{matrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{matrix} \\ = \end{matrix}$$



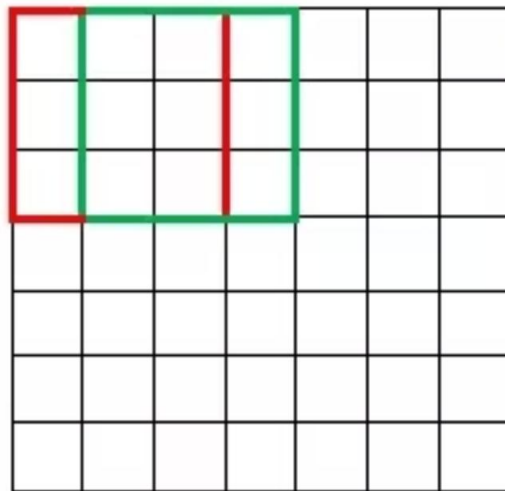
# Stride

Stride is a parameter of the neural network's filter that modifies the amount of movement over the image or video. we had stride 1 so it will take one by one. If we give stride 2 then it will take value by skipping the next 2 pixels.

$i$  -> Size of input ,  $K$  -> Size of kernel,  $S$  -> Stride

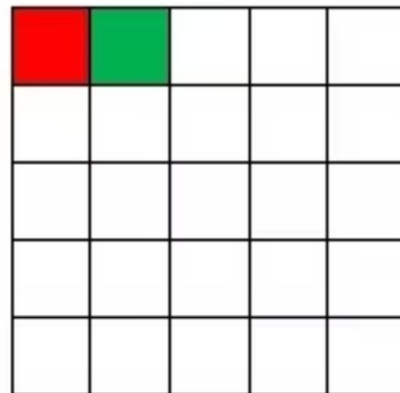
Formula  $= \lfloor \frac{i-k}{s} \rfloor + 1$

7 x 7 Input Volume

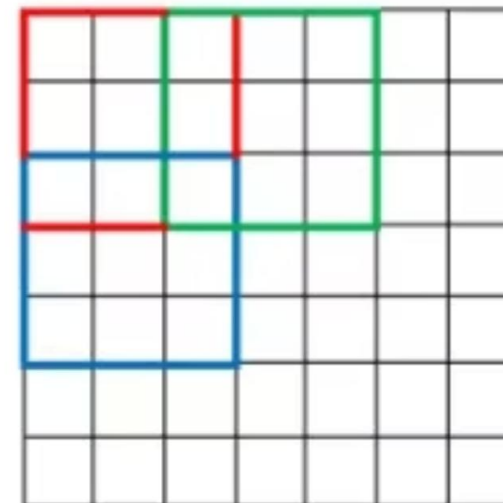


Stride 1 -->

5 x 5 Output Volume

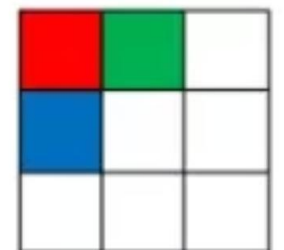


7 x 7 Input Volume



Stride 2 -->

3 x 3 Output Volume



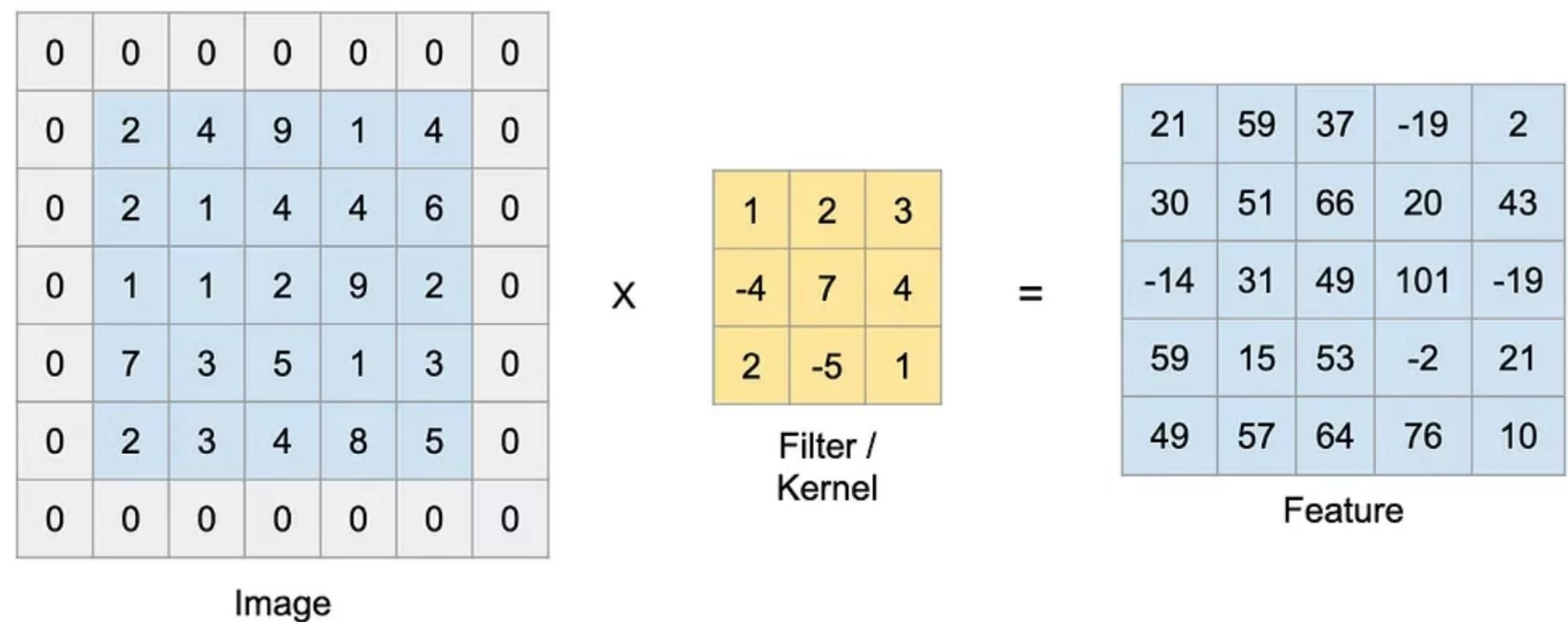


# Padding

Padding is a term relevant to convolutional neural networks as it refers to the number of pixels added to an image when it is being processed by the kernel of a CNN. For example, if the padding in a CNN is set to zero, then every pixel value that is added will be of value zero. When we use the filter or Kernel to scan the image, the size of the image will go smaller. We have to avoid that because we wanna preserve the original size of the image to extract some low-level features. Therefore, we will add some extra pixels outside the image.

$i \rightarrow$  Size of input ,  $K \rightarrow$  Size of kernel,  $S \rightarrow$  Stride,  $p \rightarrow$  Padding

Formula  $= [i - k + 2p / s] + 1$



# Convolutional Layer

- The core operation in CNNs *that extracts spatial features from images.*
- Uses filters (kernels) (e.g., 3×3, 5×5 matrices) to detect patterns such as edges, corners, and textures.
- Each filter slides over the input image and applies an element-wise multiplication followed by summation (dot product), producing a feature map.
- Mathematically, for an image  $I$  and a kernel  $K$ :

$$S(i, j) = \sum_m \sum_n I(i + m, j + n) \cdot K(m, n)$$

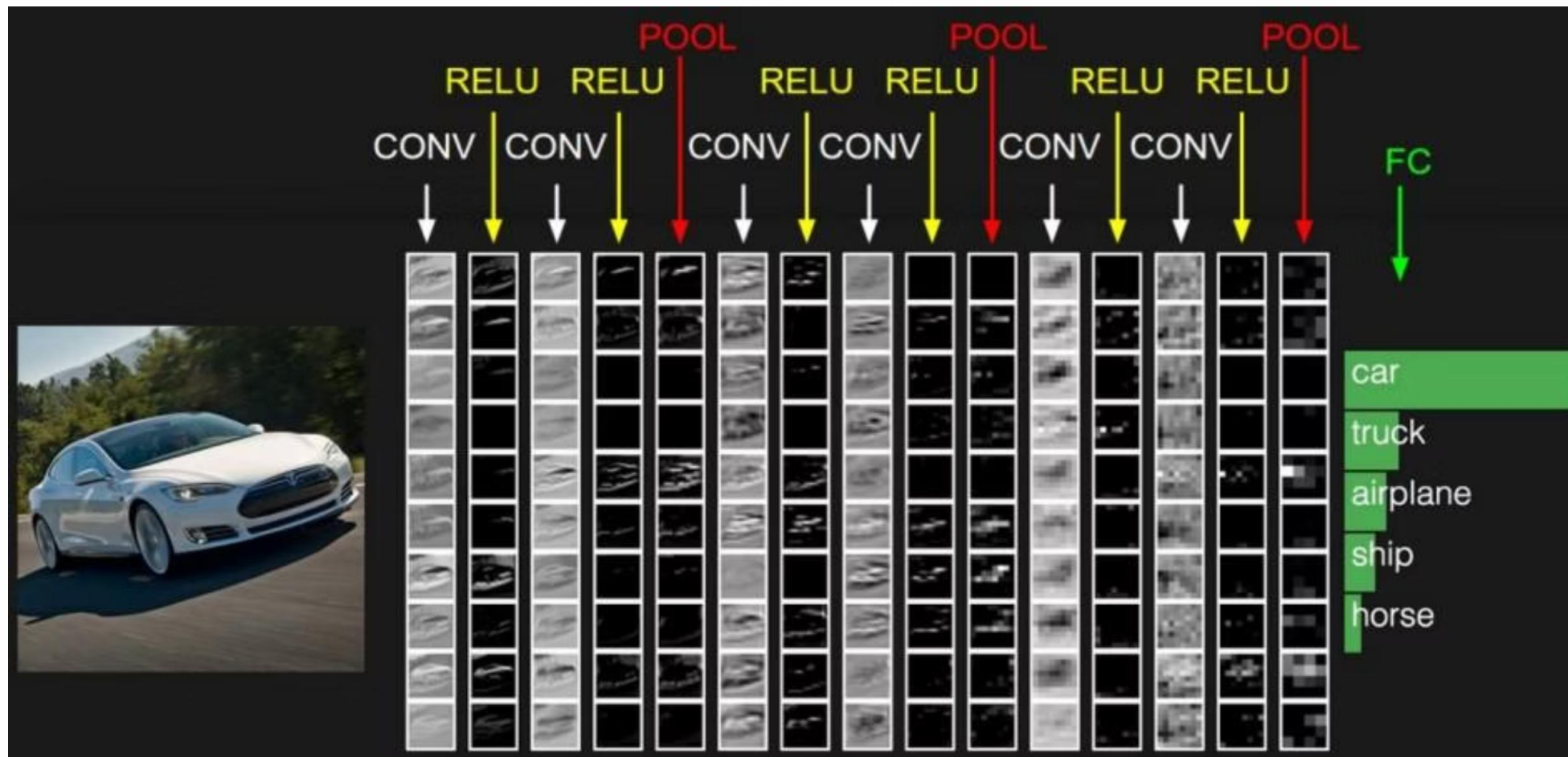
where  $S(i, j)$  is the feature map value at position  $(i, j)$ .

*Stride: Defines the step size of the filter movement.*

*Padding: Extra borders added to the input image to maintain spatial dimensions after convolution.*

# Activation Function (ReLU)

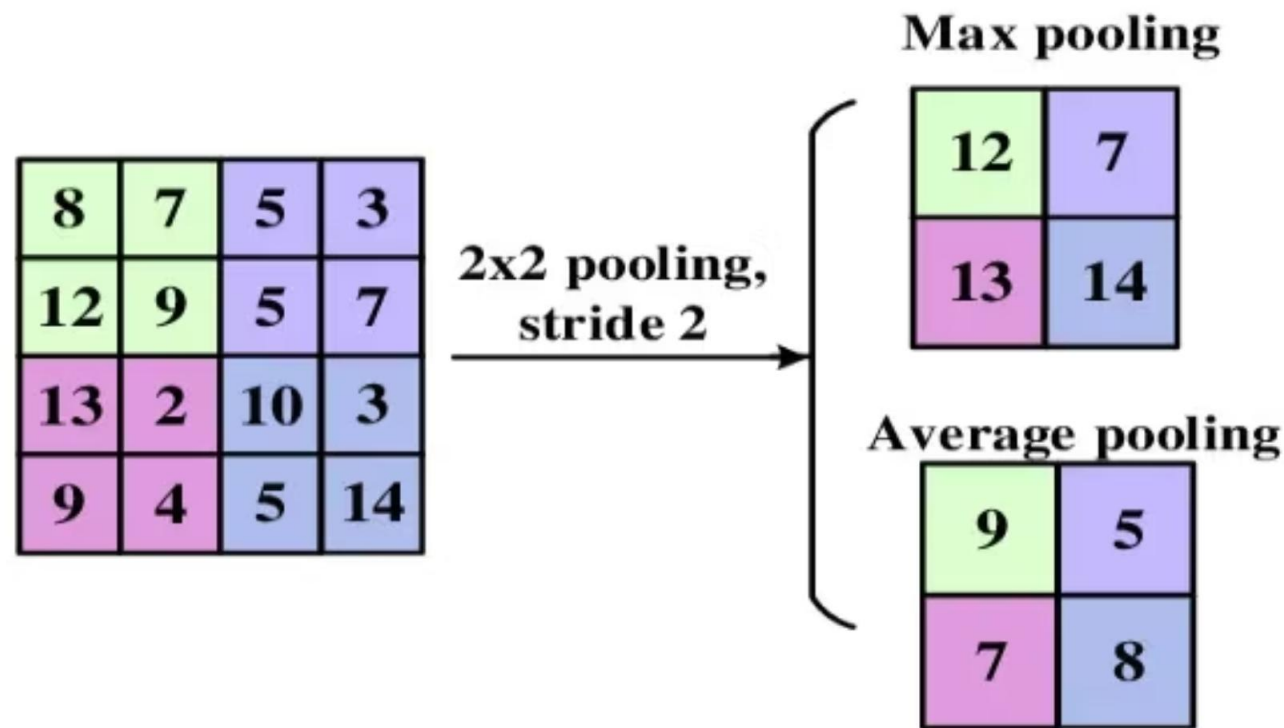
- A non-linearity function applied after convolution to introduce complexity
- ReLU (Rectified Linear Unit):  $f(x) = \max(0, x)$  replaces negative values with zero, making computations faster.



## ( Pooling Layer (Downsampling)

Reduces the spatial size (height and width) of feature maps to decrease computations and control over

- **Types:**
  - Max Pooling: Takes the maximum value from a region (e.g., 2×2 window).
  - Average Pooling: Takes the average of values in the region.
- Example: A 2×2 max pooling operation reduces a 4×4 feature map to 2×2.

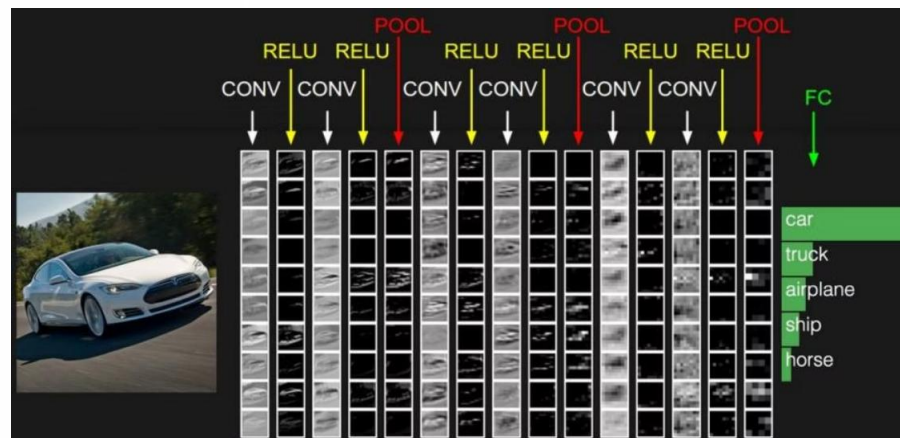




Input Image  
↓  
Convolution  
↓  
ReLU  
↓  
Pooling  
↓  
(Repeat Conv → ReLU → Pool several times)  
↓  
Flatten  
↓  
Dense (Fully Connected)  
↓  
ReLU  
↓  
Dense (Output layer)  
↓  
Softmax / Sigmoid

# Flatten

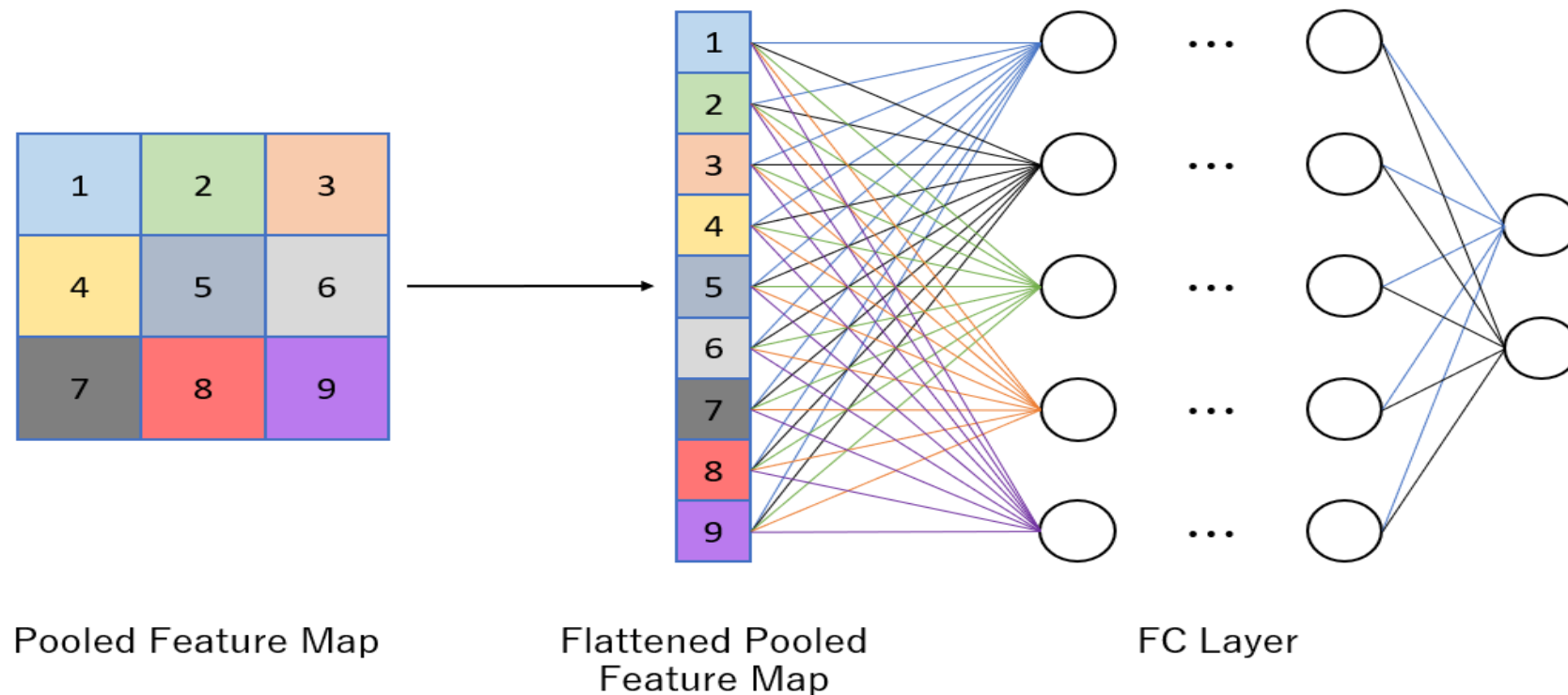
Flattening is used to convert all the resultant 2-Dimensional arrays from pooled feature maps into a single long continuous linear vector. The flattened matrix is fed as input to the fully connected layer to classify the image.



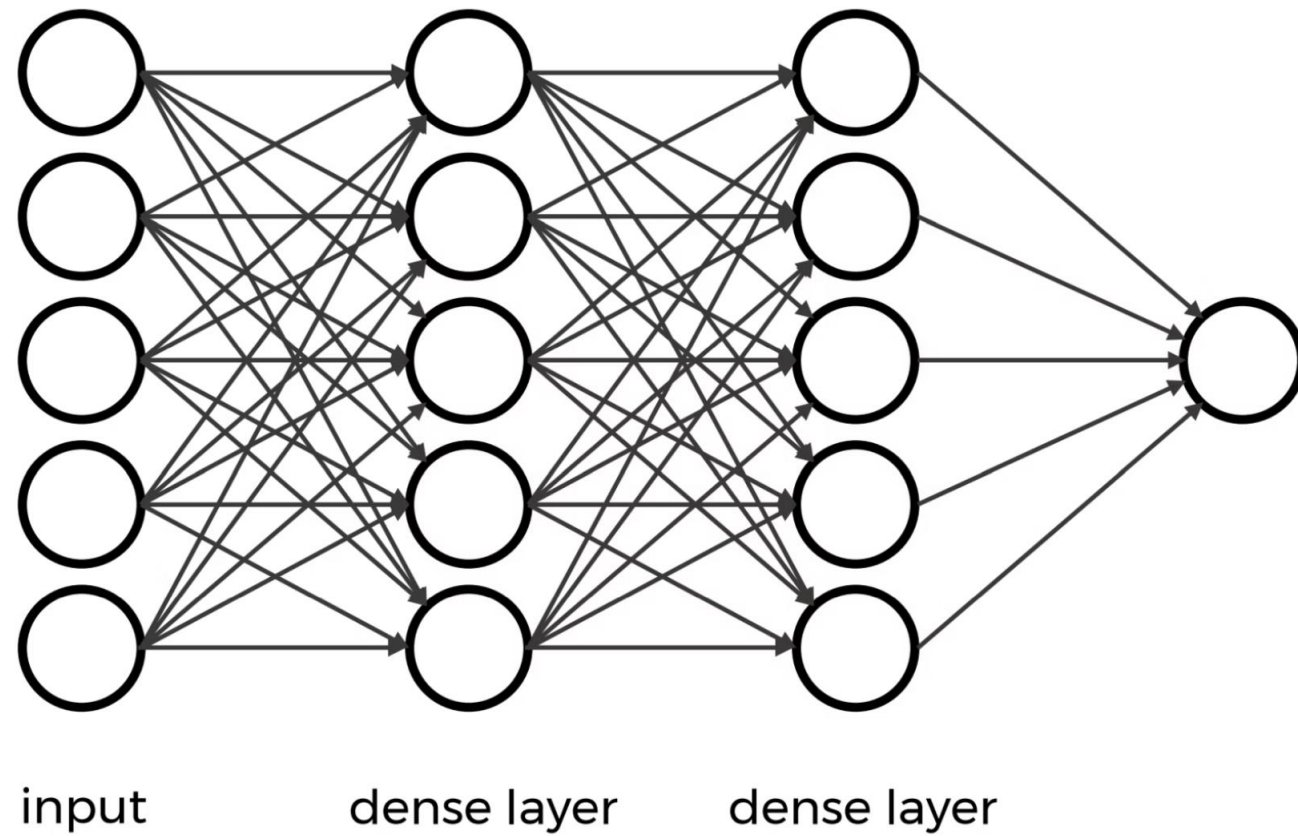
# Fully-connected layer

The Fully Connected (FC) layer consists of the weights and biases along with the neurons and is used to connect the neurons between two different layers. The next and final layer in a convolutional network is the **fully connected (FC) layer**, also known as the **dense layer**

- After feature extraction, the data is flattened into a 1D vector.
- FC layers learn high-level features and relationships between objects.
- Uses activation functions like Softmax (for classification) or Sigmoid (for binary classification).

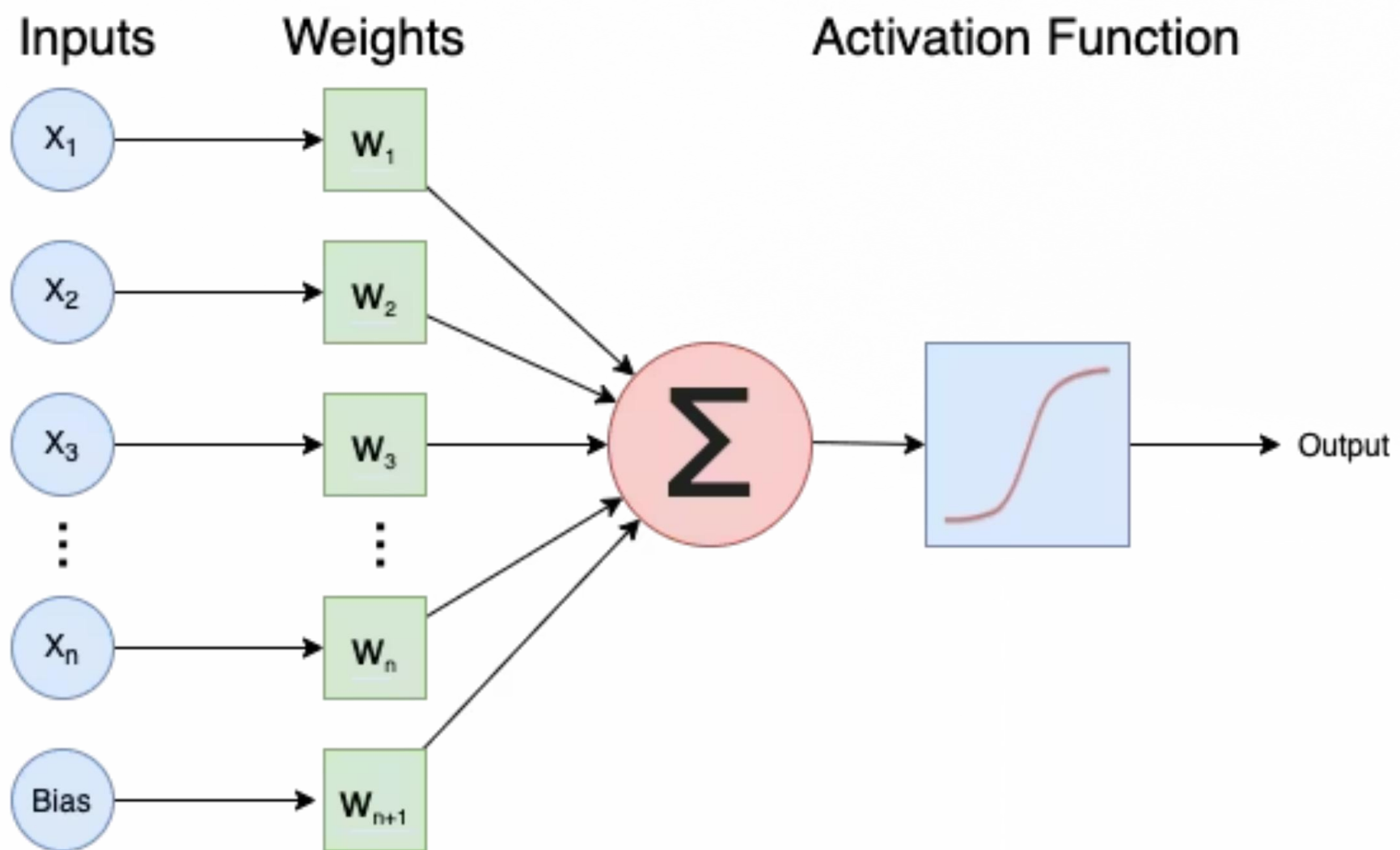


# Dense layers



A **dense** layer is a fully connected layer where each neuron receives input from every neuron in the previous layer. When connecting the layer to its input and output layers every neuron in the dense layer gets an edge (i.e. connection) to **all** of the input neurons and **all** of the output neurons.





# Activation Function

An Activation Function decides whether a neuron should be activated or not. This means that it will decide whether the neuron's input to the network is important or not in the process of prediction. There are several commonly used activation functions such as the ReLU, Softmax, tanH, and the Sigmoid functions. Each of these functions has a specific usage.

**Sigmoid** — For a binary classification in the CNN model

**tanH** - The tanh function is very similar to the sigmoid function. The only difference is that it is symmetric around the origin. The range of values, in this case, is from -1 to 1.

**Softmax**- It is used in multinomial logistic regression and is often used as the last activation function of a neural network to normalize the output of a network to a probability distribution over predicted output classes.

**ReLU**- the main advantage of using the ReLU function over other activation functions is that it does not activate all the neurons at the same time.

# Output Layer

- Produces the final prediction.
- Uses Softmax activation for multi-class classification (probabilities sum to 1).
- Uses Sigmoid activation for binary classification.

